Лабораторная работа №6. Обмен сообщениями между процессами в MPI

Поздняков Илья, группа 11-401

1. Схема обмена и теги

Программа point_to_point.c использует только MPI_Send и MPI_Recv. Процесс 0 отправляет процессу 1 строку «Hello from rank 0», процесс 1 принимает её, печатает и отвечает «Message received». В каждом вызове явно заданы буфер, число элементов (64), тип MPI_CHAR, ранг получателя или ожидаемого отправителя и тег; после приёма ранг и тег отправителя печатаются из MPI_Status. Теги: **TAG_MESSAGE = 1** — сообщение от 0 к рабочему процессу, **TAG_REPLY = 2** — ответ обратно, **TAG_PING = 3** — обмен в замерах времени. Порядок «Send → Recv» у процесса 0 и «Recv → Send» у процесса 1 исключает взаимную блокировку. Если процессов меньше двух, процесс 0 печатает понятное сообщение и все процессы завершаются через MPI_Finalize.

```
Процесс 0                                    Процесс 1
  |-- MPI_Send(msg, ..., dest=1, TAG_MESSAGE) -->|
  |                                               |-- MPI_Recv(msg, ..., source=0, TAG_MESSAGE)
  |<- MPI_Send(reply, ..., dest=0, TAG_REPLY) ---|
  |-- MPI_Recv(reply, ..., source=1, TAG_REPLY)  |
```

Доп. часть: процесс 0 в цикле по рангам r = 1…P−1 отправляет каждому «Task for rank r», затем в цикле принимает ответ именно от ранга r (source = r, тег TAG_REPLY) и проверяет, что ответ «Rank r got: Task for rank r» пришёл от правильного ранга.

2. Сборка и запуск

`mpicc -O2 -Wall -Wextra point_to_point.c -o point_to_point` — без предупреждений.
`mpiexec -n 2 ./point_to_point` — базовый обмен и обмен со всеми; `mpiexec -n 4 ./point_to_point`, `-n 8` — несколько процессов; `mpiexec -n 2 ./point_to_point time` — замер времени.
Стенд: Intel Xeon @ 2.10 ГГц (виртуальная машина), 2 ядра, Open MPI 4.1.6, GCC 13.3.0. Ядер всего 2, поэтому запуски с 4 и 8 процессами выполнялись с флагом `--oversubscribe` (процессов больше, чем ядер).

3. Результаты

Табл. 1. Базовый обмен, 2 процесса (3.1)

Запуск	Число процессов	Сообщение принято?	Ответ получен?	Программа завершилась?
1	2	да	да	да
2	2	да	да	да
3	2	да	да	да

Табл. 2. Обмен процесса 0 со всеми процессами (3.2)

Число процессов P	Отправлено сообщений	Получено ответов	Все ответы получены?
2	1	1	да
4	3	3	да
8	7	7	да

Табл. 3. Размер сообщения, обмен 0 ↔ 1 (3.3): 10 пробных обменов, затем замер; среднее по 5 запускам

Размер, байт	Число повторений	Среднее время обмена «туда и обратно», с	Скорость, ГБ/с
1	10000	$6.83 \cdot 10^{-7}$	0.003
1024	10000	$1.55 \cdot 10^{-6}$	1.32
65536	1000	$2.35 \cdot 10^{-5}$	5.57
1048576	100	$2.25 \cdot 10^{-4}$	9.34

4. Вывод

Все три запуска с двумя процессами завершились без зависаний: процесс 1 получил исходное сообщение, процесс 0 — ответ; при P = 2, 4, 8 процесс 0 отправил P − 1 сообщений и получил все P − 1 ответов от правильных рангов. В MPI процессы адресуются **рангами** внутри коммуникатора: у MPI_COMM_WORLD ранги идут от 0 до P − 1, их узнают через MPI_Comm_rank, а в MPI_Send указывается ранг получателя (dest), в MPI_Recv — ранг ожидаемого отправителя (source). **Тег** — целое число, по которому получатель отличает сообщения одного и того же отправителя: Recv принимает только сообщение с совпадающими коммуникатором, отправителем и тегом, поэтому сообщение (TAG_MESSAGE) и ответ (TAG_REPLY) не перепутаются. Время обмена складывается из задержки и передачи данных: сообщения 1–1024 байта занимают около 1 мкс (преобладает задержка), а для 1 МБ время растёт почти пропорционально размеру и скорость достигает ≈ 9 ГБ/с. Процессы работали на одной машине, поэтому обмен шёл через общую память; по сети задержка была бы заметно больше.